

design-md 스킵 설명

`design-md` 는 Figma나 기존 UI에서 디자인 시스템 근거를 수집해 `DESIGN.md` 와 토큰 산출물을 만드는 스킵이다. 이 프로젝트의 핸드오프 흐름에서는 가장 먼저 실행되며, 이후 `components` 와 `figma-implement` 가 따를 디자인 기준선을 만든다.

담당 범위

- `DESIGN.md` 생성 또는 갱신
- Tailwind v4 `@theme` CSS 산출물 생성
- Tokens Studio global token-set JSON 생성
- Figma 기반 handoff mode에서 `COMPONENTS.md` 초안 생성
- 반응형 breakpoint를 `## Layout` prose에 문서화
- shadcn/ui 사용 시 DESIGN.md token ↔ shadcn CSS 변수 정렬 매핑 보고
- `DESIGN_SPEC.md` reference 갱신 요청 처리

이 스킵은 디자인 토큰과 디자인 시스템 문서가 중심이다. 코드 컴포넌트 구현이나 route 구현은 담당하지 않는다.

언제 실행하는가

- 프로젝트 시작 시 디자인 시스템 기준을 처음 만들 때
- 색상, 타이포그래피, spacing, radius, component token 같은 foundation 값이 바뀔 때
- breakpoint 정책이 추가되거나 바뀔 때
- Figma Foundations 또는 Design System 페이지에서 토큰 산출물을 다시 뽑아야 할 때
- 개발자가 Tailwind theme CSS 또는 Tokens Studio JSON을 새로 받아야 할 때
- 구현 전 handoff 패키지(`DESIGN.md`, token artifacts, `COMPONENTS.md`)를 준비해야 할 때

실행하지 않는 경우

- 새 Figma 컴포넌트 하나만 `COMPONENTS.md` 에 등록하면 되는 경우: `components` 를 실행한다.
- 완성된 제품 페이지 frame을 코드로 구현하는 경우: `figma-implement` 를 실행한다.
- 단순 코드 컴포넌트 구현만 필요한 경우: `components` 로 계약을 확인한 뒤 별도 구현 작업으로 처리한다.

필요한 입력

기본 입력:

- Figma Foundations, Design System, Variables, Styles, Components 페이지 URL

- 또는 기존 `DESIGN.md`, token JSON, Tailwind config, CSS variables, brand guidelines
- 산출물을 저장할 프로젝트 경로

handoff mode 입력:

- Figma URL
- 이후 구현 의도, 예: "이 디자인 시스템을 코드 구현에 사용할 수 있게 준비해줘"
- 선택적으로 코드베이스 경로

Spec update 입력:

- `refresh DESIGN_SPEC.md`
- `sync the DESIGN.md spec`
- `$design-md update`

실행 요청 예시

```
File 1 Foundations URL: <figma-url>
design-md 스킴 실행.
DESIGN.md, tailwind.theme.css, tokens.studio.global.json을 만들어줘.
본문은 한국어로 작성하고 @google/design.md lint까지 확인해줘.
```

handoff mode 예시:

```
File 1 Design System URL: <figma-url>
이 디자인 시스템을 개발 핸드오프용으로 준비해줘.
DESIGN.md, tailwind.theme.css, tokens.studio.global.json, COMPONENTS.md를 생성해줘.
```

spec update 예시:

```
$design-md update
bundled DESIGN_SPEC.md를 최신 공식 spec으로 갱신해줘.
```

실행 순서

1. `design-md/references/DESIGN_SPEC.md`를 읽어 현재 schema와 section order를 확인한다.
2. Figma 또는 요청받은 source에서 디자인 근거를 수집한다.
3. 프로젝트 컨텍스트를 감지한다(`components.json`로 shadcn 여부, 기존 breakpoint 셋업).
4. YAML frontmatter에 machine-readable token을 작성한다.
5. Markdown body에 한국어 디자인 가이드를 작성한다. 반응형이면 `## Layout`에 breakpoint 값과 reflow 규칙을 문서화한다.
6. `npx @google/design.md lint DESIGN.md`를 실행한다.

7. lint 실패 시 `DESIGN.md` 를 수정하고 다시 검증한다.
8. lint 통과 후 Tailwind v4 CSS를 export한다.
9. 같은 YAML token source에서 Tokens Studio JSON을 만든다.
10. token artifact validator로 Tailwind와 Tokens Studio 산출물을 검증한다.
11. breakpoint 값과 그 위치(Layout prose, 커스텀이면 `globals.css`)를 보고한다. shaden 감지 시 token ↔ shaden 변수 매핑도 보고한다.
12. handoff mode라면 `COMPONENTS.md` 도 생성 또는 갱신한다.

출력되는 결과

기본 산출물:

파일	소비자	역할
<code>DESIGN.md</code>	에이전트, 개발자	디자인 시스템의 canonical source
<code>tailwind.theme.css</code>	개발자, 앱	Tailwind v4 <code>@theme</code> token CSS
<code>tokens.studio.global.json</code>	디자이너	Tokens Studio JSON View에 붙여 넣는 token-set

handoff mode 추가 산출물:

파일	소비자	역할
<code>COMPONENTS.md</code>	에이전트, 개발자	컴포넌트 재사용 및 신규 생성 규칙 초안

DESIGN.md 작성 규칙

- YAML frontmatter는 `DESIGN_SPEC.md` schema를 따른다.
- Markdown body는 한국어로 작성한다.
- 섹션 순서는 Overview, Colors, Typography, Layout, Elevation & Depth, Shapes, Components, Do's and Don'ts 순서를 따른다.
- `## Components` 에는 component reuse policy만 둔다.
- 코드 경로, component tree, 구체적인 mapping detail은 `COMPONENTS.md` 에 둔다.
- 근거가 있는 값은 추측해서 바꾸지 않는다.
- 근거가 부족하면 conservative하게 추론하되, 추론임을 문맥상 알 수 있게 실용적으로 쓴다.

반응형 breakpoint 처리

design.md spec에는 breakpoint token group이 없다. frontmatter schema는 `colors`, `typography`, `rounded`, `spacing`, `components` 만 정의한다. 따라서 `breakpoints:` 같은 top-level group을 만들지 않는다(spec 밖이라 lint 실패 가능).

반응형 의도는 다음 우선순위로 처리한다.

1. **## Layout prose (반응형이면 필수):** breakpoint 값(예: `sm 640 / md 768 / lg 1024 / xl 1280`)과 breakpoint별 reflow 규칙(열 수, container max-width, stack vs row)을 한국어로 명시한다. spec이 지원하는 정식 위치다.
2. **spacing token (grid 단위에 한해 선택):** spec이 허용하는 descriptive key로 `container-lg`, `gutter`, `margin` 같은 layout 단위를 둘 수 있다. viewport breakpoint를 spacing으로 위장해 넣지 않는다 (팀이 Tokens Studio export를 명시적으로 원할 때만).
3. **앱 측 breakpoint CSS (`tailwind.theme.css` 에는 금지):** Tailwind v4는 `--breakpoint-*` 를 읽지만, `tailwind.theme.css` 는 "fresh export와 일치" 검증은 받는 export 산출물이고 exporter는 `--breakpoint-*` 를 만들지 않는다. 커스텀 breakpoint는 앱의 `globals.css` (Tailwind를 import하는 stylesheet)에 `--breakpoint-*` 로 선언하도록 개발자에게 보고한다.

보고 시 문서화한 breakpoint 값과, breakpoint가 `## Layout prose`(및 필요하면 `globals.css`)에 있고 `tailwind.theme.css` 에는 없다는 점을 명시한다.

shadcn/ui token 정렬

코드베이스가 shadcn/ui를 쓰면(루트 `components.json` + `components/ui` shadcn 파일) token과 shadcn의 CSS 변수 theming이 충돌하지 않게 정렬한다.

- shadcn은 `--background`, `--foreground`, `--primary`, `--primary-foreground`, `--muted`, `--border`, `--ring`, `--radius` 같은 의미 변수를 읽는다.
- DESIGN.md token을 이 변수에 매핑해 병렬·충돌 색 체계를 만들지 않는다. `tailwind.theme.css` (token)를 source로 두고 shadcn 변수를 같은 값에서 파생시키는 방향을 권장한다.
- 매핑(DESIGN.md token ↔ shadcn 변수)을 보고해 개발자가 `globals.css` 에 wiring하게 한다. 직접 통합 요청이 없으면 shadcn `globals.css` 변수 block을 자동으로 덮어쓰지 않는다.
- 어떤 shadcn 컴포넌트가 있고 어떤 게 커스텀인지의 상세 기록은 `components` 스کیل이 관리하는 `COMPONENTS.md` 룩이다. `design-md` 는 token 정렬만 담당한다.

Tailwind 산출물 규칙

기본은 Tailwind v4 CSS다.

```
npx @google/design.md export --format css-tailwind DESIGN.md
```

출력 파일은 기본적으로 `tailwind.theme.css` 이며, `@theme { ... }` block과 `--color-*`, `--font-*`, `--text-*`, `--radius-*`, `--spacing-*` 같은 Tailwind theme variable namespace를 포함해야 한다.

`--breakpoint-*` 와 shadcn `--primary` 류 변수는 `tailwind.theme.css` 에 넣지 않는다. 그래야 이 파일이 fresh export와 동일하게 유지된다. 해당 변수들은 앱의 `globals.css` 에 둔다.

Tailwind v3 JSON은 사용자가 명시적으로 요청할 때만 만든다.

```
npx @google/design.md export --format json-tailwind DESIGN.md
```

Tokens Studio 산출물 규칙

- 기본 파일명은 `tokens.studio.global.json` 이다.
- top-level에 `$metadata`, `$themes`, `global` wrapper를 넣지 않는다.
- token set 내용만 top-level에 둔다.
- 새 파일은 W3C DTCG 형식(`$type`, `$value`)을 우선한다.
- token reference는 `{colors.primary}` 처럼 같은 token-set 내부 경로로 쓴다.
- `{global.colors.primary}` 처럼 token-set prefix를 붙이지 않는다.

검증 방법

`DESIGN.md` :

```
npx @google/design.md lint DESIGN.md
```

Tailwind와 Tokens Studio artifact:

```
node design-md/scripts/validate-token-artifacts.mjs \  
  --design DESIGN.md \  
  --tailwind tailwind.theme.css \  
  --tokens-studio tokens.studio.global.json
```

Tailwind v3 JSON을 검증할 때:

```
node design-md/scripts/validate-token-artifacts.mjs \  
  --design DESIGN.md \  
  --tailwind design.tokens.tailwind.json \  
  --tailwind-format json \  
  --tokens-studio tokens.studio.global.json
```

다른 스킴과 연계하는 방법

`components`와의 관계:

- `design-md`는 component styling policy와 token 기준을 만든다.
- `components`는 실제 Figma component와 code component mapping을 `COMPONENTS.md`에 누적 관리한다.
- handoff mode에서 `design-md`가 `COMPONENTS.md` 초안을 만들 수 있지만, 이후 컴포넌트 추가/수정의 owner는 `components`다.

`figma-implement`와의 관계:

- `figma-implement`는 `DESIGN.md`, `tailwind.theme.css`, `COMPONENTS.md`를 소비한다.

- `figma-implement` 실행 전에 `tailwind.theme.css` 가 앱의 Tailwind entry stylesheet에 import되어 있어야 한다.
- `figma-implement` 는 hardcoded color/font/spacing 대신 `design-md` 가 만든 token을 사용해야 한다.

실패 또는 중단 기준

- Figma source가 모호하면 target file, page, frame, selection을 물어봐야 한다.
- 기존 Tailwind config나 Tokens Studio JSON과 충돌하면 무리하게 덮어쓰지 않는다.
- lint 또는 artifact validation이 실패한 상태에서는 완료를 주장하지 않는다.
- spec update 요청이 아닌데 `DESIGN_SPEC.md` 를 갱신하지 않는다.

운영상 주의점

- 이 프로젝트의 `design-md/SKILL_V2.md` 는 handoff mode와 함께 반응형 breakpoint 처리, shadcn token 정렬을 포함한다.
- 현재 로컬 설치본의 `design-md` 스킬과 프로젝트 버전이 다를 수 있으므로, 실제 실행 전 설치된 skill 파일이 최신인지 확인해야 한다.
- breakpoint와 shadcn 변수는 `tailwind.theme.css` 가 아니라 앱 `globals.css` 에서 wiring한다.
- `COMPONENTS.md` 의 상세 mapping은 `components` 스킬이 계속 관리해야 한다.